

PrayogX at 17, 100, 1,000 and 10,000

What the architecture already survives, what it does not, and the order in which that matters. Every figure below is measured from the repository at commit `dc41aec`, not estimated.

COMMIT	FEED	SIMULATIONS	PUBLISHED/SIM	APK PAYLOAD	SUITES
dc41aec	33505a9321bd	17	145.4 KB	41.8 KB	14 / 484

There is nothing in the MUST FIX NOW column. The content architecture is sound and scales further than the business plausibly needs. The single most important number in this audit is **41.8 KB**: the entire web payload inside the Android APK. It contains no simulation and no catalogue, so the app binary does not grow when the library does — at 17 simulations or at 10,000. That property is what makes every other scaling question a content question rather than a release question.

Five findings are worth doing before 100. None is architectural. Four of them are about what you cannot currently see — errors, usage, and one genuine data-loss exposure — and one is a forward-compatibility latch that costs an hour now and cannot be retrofitted into an app already on a student's phone.

The hard ceiling is GitHub Pages at roughly 7,000 simulations, and moving past it is a two-line change to a config, not a migration. That is the architecture paying for itself.

THE NUMBERS

Measured per simulation, projected linearly

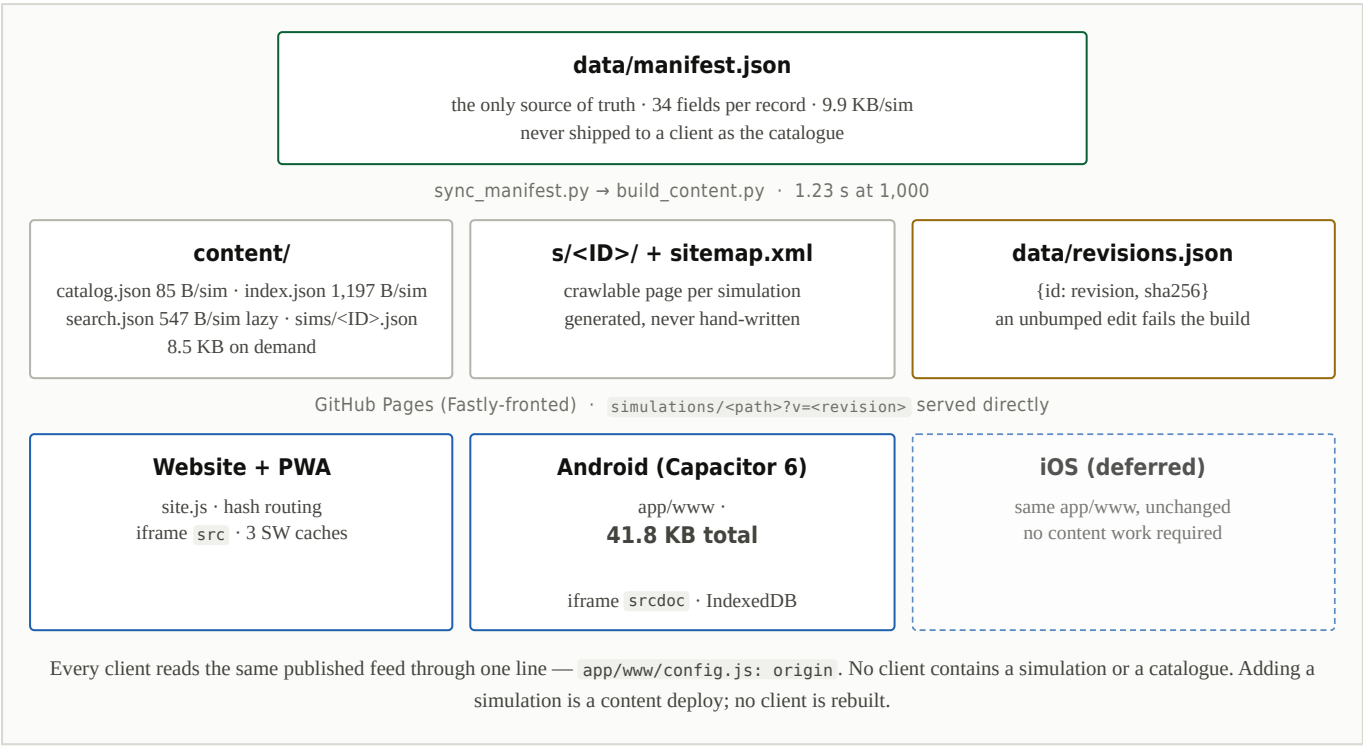
COMPONENT	PER SIM	17	100	1,000	10,000
catalog.json (every launch)	85 B	1.4 KB	8.5 KB	85 KB	850 KB
index.json (startup)	1,197 B	20 KB	120 KB	1.20 MB	12.0 MB
search.json (lazy)	547 B	9 KB	55 KB	547 KB	5.5 MB
content/sims/ (on demand)	8.5 KB	144 KB	847 KB	8.5 MB	85 MB
simulations/ (the content)	109.4 KB	1.86 MB	10.9 MB	109 MB	1.09 GB
s/ crawlable stubs	5.8 KB	98 KB	579 KB	5.8 MB	58 MB
data/manifest.json + .js	19.7 KB	335 KB	1.97 MB	19.7 MB	197 MB
data/revisions.json	132 B	2.2 KB	13 KB	132 KB	1.3 MB
Published site total	145.4 KB	2.5 MB	14.5 MB	145 MB	1.45 GB
Android APK web payload	0 B	41.8 KB	41.8 KB	41.8 KB	41.8 KB

Published bytes by component. Per-simulation figures are real bytes on disk divided by 17, not estimates. "Site total" is everything GitHub Pages serves.

WHAT	17	1,000	READING
Startup transfer	55 KB	1.26 MB	Index, not manifest — the split works
First paint	—	698 ms	Fine
Cards in the DOM	17	24	Pagination holding
Filter a chapter	—	419 ms	Acceptable
Full-text search	—	1,423 ms	The wall. Linear scan, every keystroke
build_content.py	0.038 s	1.23 s	Linear; ~12 s at 10,000
json.load(manifest)	—	0.81 s	8.4 MB file; ~8 s at 10,000
Full test suite	181 s	—	14 suites, 484 checks
Behaviour measured in a headless browser against a synthetic 1,000-simulation catalogue, and pipeline timings at both scales.			

ARCHITECTURE

One source, one derive step, four clients



FINDINGS

Twenty-five areas, classified

A must fix now **B** before 100 **C** before 1,000 **D** can wait **E** no change required

F1 Nothing reports an error, so you cannot know a simulation broke**B**

CURRENT	No <code>window.onerror</code> , no <code>unhandledrejection</code> handler, no reporting anywhere in <code>site/site.js</code> , <code>app/www/app.js</code> or <code>sw.js</code> . Seven catch blocks swallow their error silently.
EVIDENCE	<code>grep -c "window.onerror unhandledrejection" → 0</code> in all three. Seven matches for an empty <code>catch (e) {}</code> .
RISK	A student hits a broken simulation, sees a blank frame, and closes the app. You find out when someone mentions it in class, or never. This already cost a session: the "No connection" message hid a 404 for hours.
AT SCALE	Constant probability per simulation, so linear in library size. At 17 you would hear about it. At 100 you will not.
DO	A global handler in both shells that shows the student something honest and records the last error where you can read it later. No backend needed for step one.
NOW? · IOS? · EFFORT	Not urgent · yes, same file · Low

F2 The exam PDFs exist on one Mac and nowhere else**B**

CURRENT	<code>papers/</code> is 5.7 MB and deliberately gitignored so it never reaches the public site. It is therefore in no backup at all.
EVIDENCE	<code>du -sh papers → 5.7M; git ls-files papers wc -l → 0</code> . Everything else is on GitHub.
RISK	Disk failure loses the source material every future simulation is derived from. Recoverable in principle — they are public exam papers — but the curated working set, and any annotation, is not.
AT SCALE	Grows with every paper you add. The exposure compounds.
DO	A private backup — a second private repo, or any encrypted cloud copy. Explicitly <i>not</i> the public repository; keeping them out of the deploy is correct and should stay.
NOW? · IOS? · EFFORT	Not urgent · no · Low

F3 `schemaVersion` is written by the build and read by nobody**B**

CURRENT	<code>build_content.py:288</code> stamps <code>"schemaVersion": "1.0.0"</code> into <code>catalog.json</code> . No client checks it.
EVIDENCE	<code>grep -n schemaVersion site/site.js app/www/app.js → no matches</code> . The only hit is the generator.
RISK	An installed Android app is frozen at its install-time parser but reads a live feed. The day you make a breaking feed change, every app that has not been updated mis-parses it silently rather than saying so. This is the one finding that <i>cannot</i> be retrofitted into a copy already on a phone.
AT SCALE	Independent of library size; proportional to how many installs exist when you first need a breaking change. Do it while the install base is small.
DO	Clients compare the feed's major version to the one they were built for; on a mismatch, show "update the app" instead of rendering wrong.
NOW? · IOS? · EFFORT	Worth doing early · yes, directly · Low

F4 No usage signal, so the library grows on instinct

B

CURRENT	Zero telemetry of any kind: no analytics script, no beacon, no counter, in the shell, the app, the worker or the tools.
EVIDENCE	A grep for every common provider and <code>sendBeacon</code> across <code>site/</code> , <code>app/www/</code> , <code>index.html</code> , <code>sw.js</code> , <code>tools/</code> returns nothing.
RISK	Not a failure risk — a decision risk. You cannot tell which simulations students actually open, which is the input to deciding what to build next. It is also, today, a genuine privacy virtue worth keeping deliberately rather than by accident.
AT SCALE	At 17 you can ask a class. At 100 you cannot, and the cost of building the wrong thirty simulations is thirty simulations.
DO	The smallest thing that answers "which simulations were opened": a privacy-respecting counter. This needs a sink, not a backend, and no personal data. Decide deliberately — including deciding not to.
NOW? · IOS? · EFFORT	Not urgent · yes, shared shell · Low–Medium

F5 No Content-Security-Policy on either shell

B

CURRENT	No CSP meta in <code>index.html</code> or <code>app/www/index.html</code> . Simulation frames carry no <code>sandbox</code> attribute.
EVIDENCE	<code>grep -c Content-Security-Policy → 0</code> in both; <code>grep -c sandbox → 0</code> in <code>site.js</code> and <code>app.js</code> . Android posture is good by contrast: <code>INTERNET</code> the only permission, <code>androidScheme: https</code> , <code>allowMixedContent: false</code> , no cleartext config.
RISK	Low today and honestly so: every simulation is first-party, self-contained, and verified to load nothing external. The risk arrives the day a simulation comes from someone else — a student, a colleague, a contributor.
AT SCALE	Rises with the number of authors, not the number of simulations.
DO	A CSP meta on both shells. Be careful with <code>sandbox</code> on the frames: the Top control and the anchor shim both need same-origin access to <code>contentWindow</code> , and <code>allow-scripts allow-same-origin</code> together on same-origin content buys little. CSP is the useful half.
NOW? · IOS? · EFFORT	Not urgent · yes, shared shell · Low

● BEFORE 1,000 SIMULATIONS

4 findings

F6 Search is a linear scan on every keystroke

C

CURRENT	<code>search.json</code> is one blob of 547 B per simulation, fetched lazily on the first query, then scanned in full per keystroke.
EVIDENCE	Measured against a 1,000-simulation catalogue: 1,423 ms per search, against 419 ms to apply a filter and 698 ms to first paint.
RISK	The first thing a student will complain about. Invisible at 17, fine to roughly 300, sluggish by 500.
AT SCALE	Linear: ~7 s at 5,000. Also 547 KB downloaded at 1,000, 5.5 MB at 10,000.
DO	Build an inverted index at publish time — <code>token → id</code> list — instead of scanning at query time. It shrinks the download as well. Do not do it yet; it buys nothing at your size and it is real work.
NOW? · IOS? · EFFORT	No · yes, both clients · Medium

F7 One addition makes every client re-download the whole index

C

CURRENT	The feed version is a hash of all card records. Adding one simulation moves it, so both clients fetch <code>index.json</code> in full.
EVIDENCE	<code>index.json</code> is 1,197 B/sim monolithic → 1.20 MB at 1,000. <code>catalog.json</code> is fetched cache: "no-store" on every launch and every resume — 85 KB at 1,000, 850 KB at 10,000, on mobile data.
RISK	Publishing weekly at 1,000 simulations costs every student 1.2 MB per week for one new card.
AT SCALE	12 MB per addition at 10,000. This is the finding that decides whether 10,000 is pleasant.
DO	Shard the index by year or subject so only the touched shard moves, or ship a delta keyed on the previous version. The catalogue's revisions map can also move to a separate, smaller poll.
NOW? · IOS? · EFFORT	No · yes, both clients · Medium

F8 The authoring manifest is one file that becomes unopenable

C

CURRENT	A single <code>data/manifest.json</code> with 34 fields per record, plus a byte-identical <code>manifest.js</code> mirror for the <code>file://</code> fallback.
EVIDENCE	9,858 B/sim → 167 KB at 17, 9.9 MB at 1,000, 98 MB at 10,000 . <code>json.load</code> alone takes 0.81 s at 1,000. Both files are published, so the pair costs 19.7 KB per simulation of the deploy.
RISK	An authoring problem, not a runtime one: editors stall, git diffs become unreadable, and a merge conflict in a 98 MB JSON file is not resolvable by hand.
AT SCALE	Fine to 1,000. Painful beyond it.
DO	Shard the authoring source per year or subject and concatenate at build time. The derived feed does not change shape at all, so no client is affected. Separately: reconsider whether both fallback copies still need publishing.
NOW? · IOS? · EFFORT	No · no — authoring only · Low–Medium

F9 GitHub Pages runs out at roughly 7,000 simulations

C

CURRENT	Static hosting on GitHub Pages, deployed by a custom Actions workflow with a deny-list tar and a pre-deploy validation gate.
EVIDENCE	Published site is 145.4 KB per simulation → 2.5 MB today, 145 MB at 1,000, 1.45 GB at 10,000 . GitHub's published limits: 1 GB site size (hard) , 100 GB/month bandwidth (soft), 10-minute deployment timeout. Every publish re-uploads the whole site.
RISK	None until roughly 7,000. Bandwidth is not the binding constraint — 100 GB is about a million simulation opens per month at 109 KB each.
AT SCALE	Fine at 100 and 1,000. The ceiling is real and arrives before 10,000.
DO	Nothing now. When it arrives, move the content host: <code>app/www/config.js</code> and <code>SITE_URL</code> in <code>build_content.py</code> are the two lines. Pages is already Fastly-fronted, so you are not giving up a CDN by staying.
NOW? · IOS? · EFFORT	No · no — one config line · Low when needed

F10 Detail records are not revision-keyed, unlike simulations

D

`content/sims/<ID>.json` is fetched at a bare URL while the simulation HTML carries `?v=<revision>`. Stale-while-revalidate corrects it on the next view, so the visible effect is a briefly out-of-date description. It is the one place the revision discipline is not applied end to end. **Low**, both clients.

F11 The offline cache evicts oldest-opened, not least-recently-used

D

`trim()` in `sw.js` deletes from the front of `cache.keys()` — insertion order, so FIFO, while the comment says LRU. A student who reopens Q1 daily still loses it once 40 newer simulations arrive. The comment is wrong either way. **Low**, web only.

F12 28% of the published site is files no client requests

D

Each simulation folder publishes `meta.json` and `question.md` beside it, and `data/` publishes both manifest copies: 36 KB per simulation of authoring material against 109 KB of actual simulation. Nothing private leaks — the workflow fails if a PDF reaches the deploy — it is weight. Note `manifest.json` is a live fallback the site fetches when the feed fails, so only the companions can simply go. **Low**, one exclude line.

F13 `status: deprecated` is validated but nothing consumes it

D

`check_library.py` enforces the enum `human_verified | draft | deprecated`, but neither client reads it. A retired simulation simply vanishes from the feed, and a student's stored copy lingers until the cap evicts it. Fine today; it becomes a content-lifecycle question when you start revising the library rather than only adding to it. **Low**.

F14 Test suite runtime is 181 s and mostly independent of library size

D

14 suites, 484 checks. The browser suites run against fixtures, so they do not grow with the library; `propagation.js` and `stalecache.js` run the real pipeline, which is 1.23 s at 1,000 and roughly 12 s at 10,000. No action. Watch it if the wall-clock passes about five minutes — that is where people stop running it. **Low**.

F15 Accounts and a paywall are already expressible without a backend

D

`access ( free | premium | pro )` and `status` are validated enums on every record, and `byAccess` is already in the catalogue counts — so a tier can be expressed in content today, with no server. Favourites and recents are per-device (`localStorage`), which is the right default. **Do not add a backend to enable this**. Add one only when you need something a static file genuinely cannot do: cross-device sync, or enforcing payment rather than labelling it. **High** when it comes, and it is not close.

F16-F25 The parts carrying the design — leave them alone**E**

F16 · Canonical source. `data/manifest.json` is the only source; everything else is derived. `check_library.py` enforces seventeen distinct failure modes — duplicate ids, id format, broken paths, missing companions, `meta.json` disagreeing with the manifest, stale counts, stale feed, stale detail records, missing crawlable pages, an unstamped worker, an unbumped revision, orphan folders. The deploy workflow re-runs the whole pipeline and refuses to publish if the generated output differs from what was committed.

F17 · Delivery. One self-contained HTML file per simulation, 109 KB average, no CDN, no fonts, no fetch, no storage. This is what makes `srcdoc`, offline rendering and the iOS story all work at once.

F18 · Revision and cache invalidation. `revisions.json` holds `{revision, sha256}`; an edit without a bump fails the build. Every simulation URL carries `?v=`. The worker's version is stamped from the feed at build time. The simulations cache is deliberately *not* version-named, so growing the library no longer evicts what a student saved.

F19 · Add once, appear everywhere. Proven on every test run, not asserted: `propagation.js` adds a real simulation to the canonical source, runs the real pipeline, and checks the website, search, the revision-tagged link, the detail page, the generated crawlable page, the PWA cache and the app all pick it up with no code change — then removes it and confirms the feed version returns to the identical string.

F20 · Android footprint. `app/www` is **41.8 KB**, four files, containing no simulation and no catalogue. The APK does not grow with the library. `production_audit.py` asserts this and `appcheck_prod.js` watches every request the shell makes.

F21 · Web performance. 24 cards in the DOM regardless of catalogue size; 698 ms first paint at 1,000.

F22 · Offline. Web: three caches, three lifetimes, LRU cap 40. App: IndexedDB keyed by id with the revision inside the record, cap 30. Worst case about 3.6 MB of stored HTML, and it does not grow with the library.

F23 · Routing. Hash routing needs no server rewrites — correct for a static host — with a generated crawlable page per simulation carrying the canonical URL, and the simulation's own file URL still working standalone.

F24 · Migration keys. Everything durable is already versioned: `localStorage` prefix `prayogx:v1`; IndexedDB `prayogx v1` store `sims`, cache `prayogx-sims-v1`. A future schema change has a clean upgrade path.

F25 · Android security posture. One permission (`INTERNET`), `androidScheme: https`, `allowMixedContent: false`, no cleartext config, `CapacitorHttp` enabled so the content host is never asked for CORS headers.

READINESS

By milestone**100 simulations****Ready today.**

Site 14.5 MB, index 120 KB, search imperceptible, APK unchanged, deploy 3 s. Nothing in the architecture strains. Do F1–F5 in this window — not because 100 forces them, but because they are cheap now and two of them (F3, F4) get harder as the install base grows.

1,000 simulations**Ready with two changes.**

Site 145 MB, first paint 698 ms, filter 419 ms — all fine. Search at 1,423 ms (F6) is the one a student notices, and the 1.2 MB index re-download on every publish (F7) is the one that costs them data. Authoring in one 9.9 MB manifest (F8) is tolerable. Everything else holds unchanged.

10,000 simulations**Needs a host move and a sharded feed.**

1.45 GB exceeds the GitHub Pages 1 GB hard limit — the ceiling arrives near 7,000. A 98 MB authoring manifest is not editable. Search would be ~14 s. None of this is architectural: the content model, the revision lock, the clients and the APK are all unchanged at 10,000. It is F6, F7, F8 and F9, in that order.

Deferred, and safely so

No architectural rework is required, and the gap did not widen while iOS was deferred. Everything platform-specific is already shared or absent:

- `app/www` — all 41.8 KB of it — is the whole client, and iOS runs the same four files.
- `capacitor.config.json` already carries its `ios` block (`contentInset: "always"`, app-bound domains off).
- `app.css` reads `env(safe-area-inset-*)` natively; the Android inset bridge in `MainActivity` is Android-only and not in the way.
- The `srcdoc` anchor fix shipped this week is in `app.js`, so iOS inherits it. Without it, iOS would have had the identical bug.
- `srcdoc`, `IndexedDB`, `localStorage` and `CapacitorHttp` all work in `WKWebView`.

What remains is work, not redesign: `npm cap add ios` on macOS, signing, artwork, a listing. Two things to know before submitting, neither structural:

- **App Store review 2.5.2 / 4.7.** The app fetches HTML and JavaScript at runtime and runs it in a web view. That is permitted — it is web content in a web view, and the app ships no executable updates — but reviewers sometimes open the conversation. Have the answer ready rather than being surprised by it.
- **WKWebView data persistence.** Safari's seven-day eviction of unused website data does not apply to an app's own `WKWebView` store, but verify it on a device before promising students that downloaded simulations stay downloaded.

Of the findings above, three touch iOS directly and are worth resolving *before* an iOS build rather than after: **F3** (version latch), **F1** (error reporting) and **F5** (CSP) — all in the shared shell, so doing them once covers both platforms.

BOTTLENECKS AND SINGLE POINTS OF FAILURE

Ranked by when they bite

ITEM	BITES AT	NATURE
Full-text search — linear scan	~300	Performance, user-visible
Index re-download on every publish	~500	Bandwidth, student data cost
Authoring manifest as one file	~2,000	Workflow, not runtime
GitHub Pages 1 GB published site	~7,000	Hard limit
GitHub Pages bandwidth 100 GB/mo	~1M opens/mo	Soft limit, not binding
GitHub Pages / one GitHub account	any time	SPOF — one host, one account, one org
The <code>origin</code> line	any time	SPOF by design; two source locations, not one
<code>papers/</code> unbacked	any time	Data loss — F2
One maintainer	any time	The real SPOF. Docs are good; the knowledge is not distributed

Two of these deserve naming plainly. The GitHub account is a single point of failure for the site, the source and the backup at once — the repository is the backup, and it lives in the same place as the thing it backs up. And the deploy re-uploads the entire site on every publish, which is invisible at 2.5 MB and a 145 MB upload at 1,000.

ROADMAP

In the order the evidence supports

- 1 **Now, before anything else ships to more students.** F2 (back up `papers/` privately) and F3 (version latch in both shells). F2 because the exposure is real today; F3 because it is the only finding that cannot be fixed retroactively on a phone that already has the app.
- 2 **Same window, cheap.** F1 (global error handler) and F5 (CSP). Both in the shared shell, both Low, both benefit iOS before it exists.
- 3 **Decide deliberately.** F4 (usage signal). The answer may legitimately be no — but make it a decision, not an omission, and write down which it was.
- 4 **iOS, when you want it.** Nothing above blocks it; steps 1–2 make it better.
- 5 **Around 300 simulations.** F6 (inverted search index). This is the first change a student will feel.
- 6 **Around 500.** F7 (shard or delta the index).
- 7 **Around 2,000.** F8 (shard the authoring manifest).
- 8 **Around 5,000, with headroom.** F9 (move the content host). Two config lines, but do it before 7,000, not at it.
- 9 **Never, unless a static file genuinely cannot do the job.** F15 (a backend).

DO NOT CHANGE

The decisions the rest depends on

- 1 **`data/manifest.json` as the only source**, with everything else derived. This is the decision every other property rests on.
- 2 **The revision lock.** `revisions.json` plus the build's refusal of an unbumped edit is what makes cache invalidation reliable rather than hopeful.
- 3 **Self-contained single-file simulations.** No CDN, no fonts, no fetch, no storage, no build step. Verified across all 17.
- 4 **The app shipping no catalogue and no simulation.** 4.1.8 KB, asserted by the audit tool and watched request-by-request by the shell suite.
- 5 **One origin line pointing every client at the content host.** It is also the escape hatch when Pages runs out.
- 6 **The deny-list deploy and the stale-content refusal.** A new directory is published by default; a feed that does not match its source is never published at all.
- 7 **The stable simulations cache name.** Hard-won; re-versioning it would silently wipe every student's offline library on the next publish.
- 8 **The current toolchain.** Capacitor 6.2.2, AGP 8.10.0, Gradle 8.11.1, `compileSdk/targetSdk 36`, `minSdk 22`. It builds, it runs on API 36, it meets the Play floor. Do not move any of it until Play forces you to.
- 9 **Test-first on the content pipeline.** 484 checks is why three separate regressions this month were caught by a suite rather than by a student.

Audited at commit `dc41aec`, feed version `33505a9321bd`, 17 simulations, working tree clean and nothing unpushed. Every per-simulation figure is real bytes on disk divided by 17; every timing is a measured run, either against the live repository or against a synthetic 1,000-simulation catalogue. Nothing in the repository was modified, and no file was created inside it, to produce this report.